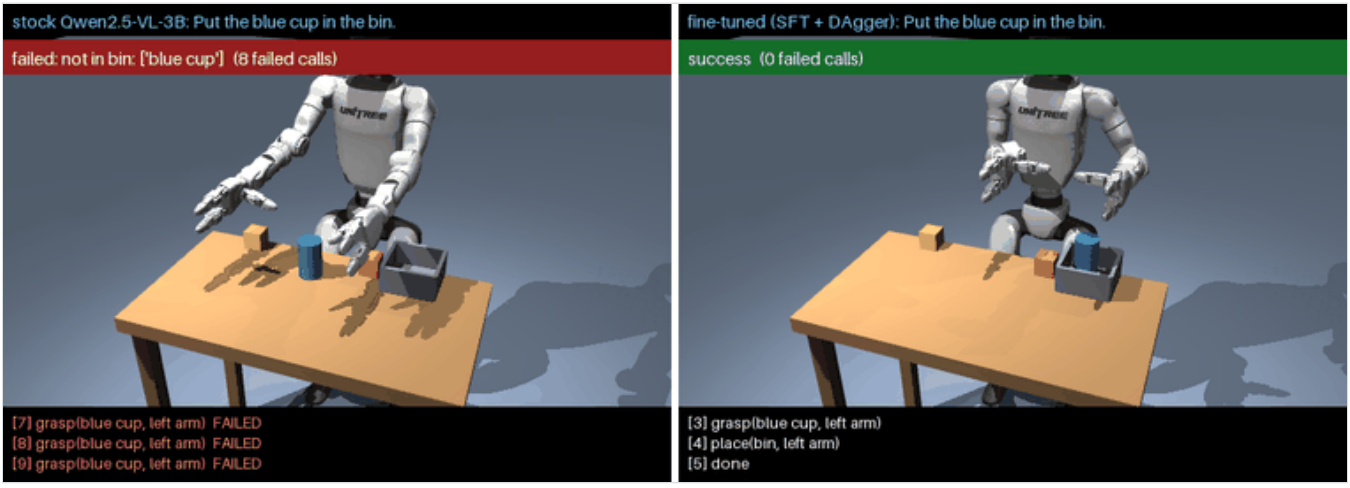
Study report · Nishanth Sundaran · 3 November 2026 · github.com/NishanthSundaran/one-brain-many-bodies/posttrain

Planner Post-Training Study Guide

How to turn a general vision-language model into a reliable robot planner, explained through one complete project: supervised fine-tuning, DAgger and GRPO on a 3B model, every result read from physics, on one 6 GB laptop GPU.

This guide is written to be studied. Each idea is explained first, then shown in the measurements from the project. Read it in order the first time; afterwards the interview section and glossary work on their own. It is the companion to the LIBERO Imitation Study Guide: that one is about learning motor control from demonstrations, this one is about teaching the planner that sits above the motor skills.



The same held-out task, "Put the blue cup in the bin", at the end of each episode. Left: the stock model grasps with the right arm, places with no arm given and retries a failing grasp until the episode stops. Right: the fine-tuned planner sets the cup down at the middle, takes it with the left arm and drops it in the bin, in five calls. The animated version is `demos/media/14_finetuned_planner.gif`.

23 → 92 held-out tasks solved out of 100, before and after post-training	**94** tasks the privileged expert solves: the ceiling for imitation
11 vs 2 tasks only DAgger solves vs only SFT solves (paired, p = 0.022)	**0 vs 0** tasks GRPO changed: an honest null result, explained in section 9

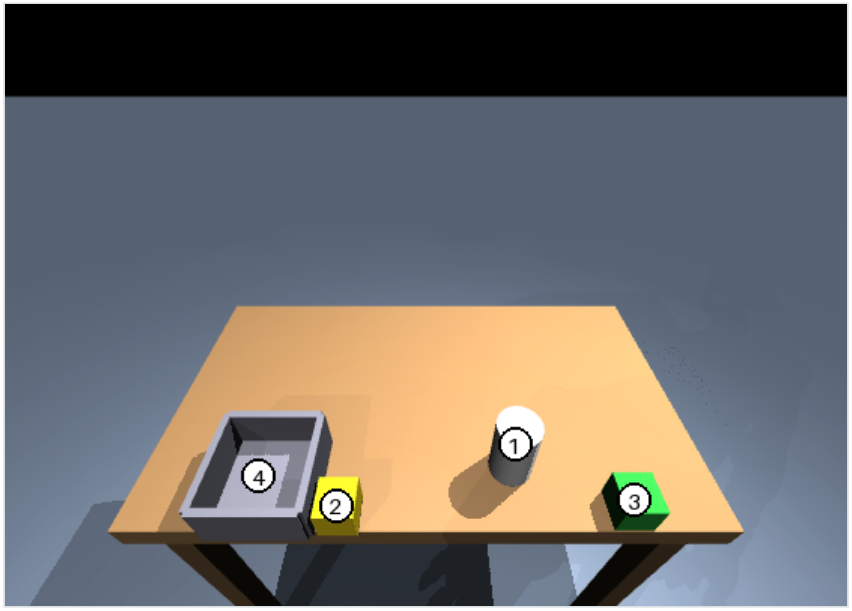
Contents

1. The problem: a language model that drives a robot 4
2. The benchmark and its expert 5
3. Core ideas you need 6
4. The pipeline, stage by stage 8
5. Evaluating fairly 10
6. What happened, step by step 11
7. Final results 12
8. How the planners fail 13
9. Why GRPO changed nothing 14
10. Lessons that transfer 14
11. Interview questions 15
12. Glossary 16
13. Further reading 18

1. The problem: a language model that drives a robot

The One Brain, Many Bodies stack controls a Unitree G1 humanoid in MuJoCo with a **hierarchy**. At the top, a vision-language model (the **planner**) looks at one camera frame and decides the next step. It does not move joints. It calls a **tool**, such as `grasp(object, arm)` or `place(target, arm)`, and deterministic skills underneath turn that call into motion: inverse kinematics, a compliant controller at about 500 Hz, and a weld that attaches the object when the hand closes. The planner runs at roughly one decision every four to five seconds.

This split is common in robotics with language models (SayCan, Code as Policies and most "LLM agent" robot demos work this way). It means the planner never has to be precise about motion, only about *decisions*: which object, which arm, where to put it, and when to stop. Those decisions are exactly where a small model is weak.



What the planner sees: the head camera frame with numbered markers (Set-of-Mark prompting). Marker 4 is the bin on the robot's left, marker 1 is a white cup on its right.

Every turn the planner receives a system prompt (the tools, the rules about arms and reach), the frame above, and a text state message. A real one from the training data:

```
TASK: Put the yellow block and the white cup in the bin.
MARKERS: {"1": {"name": "white cup", "position": [0.347, -0.119, 0.7699], "arm":
"right"},
          "2": {"name": "yellow block", "position": [0.2981, 0.1057, 0.9069],
"arm": "left"},
          "3": {"name": "green block", "position": [0.287, -0.263, 0.7449], "arm":
"right"},
          "4": {"name": "bin", "position": [0.32, 0.21, 0.72], "arm": "left"}}
```

```
ACTION HISTORY: grasp({"object": "yellow block", "arm": "left"}) -> OK
PREVIOUS ACTION RESULT: {"tool": "grasp", "ok": true, "arm": "left", "holding":
"yellow block"}
```

and must answer with exactly one JSON tool call:

```
{"thought": "The left hand holds the yellow block and the bin is on the left
side.",
 "tool": "place", "args": {"target": "bin", "arm": "left"}}
```

The hard part of this task is the white cup. It is on the right, the bin is on the left, and an arm cannot reach across the body. The only solution is a **handover**: the right arm sets the cup down in the middle of the table, then the left arm picks it up and drops it in the bin. A planner has to work that out from coordinates and the "arm" hints, and the stock model almost never does.

---

2. The benchmark and its expert

A fine-tuning study is only as good as its evaluation, so the first piece of work was a task suite where success is unambiguous and every task is solvable.

Six task families

family	example command	what it tests	eval tasks
bin	Put the blue cup in the bin.	arm choice, handover when the object is on the far side	28
two_bin	Put the blue cup and the red block in the bin.	sequencing two objects, often one handover	16
next_to	Put the white cup next to the red block.	choosing a free spot 5 to 16 cm from another object	25
side	Move the blue cup to the left side of the table.	cross-table move, always a handover	16
absent	Put the pink block in the bin.	noticing the object is not there and doing nothing	4
already	Put the white cup in the bin. (it already is)	calling done at once instead of "fixing" a solved task	11

Success comes from physics, not from the model

At the end of an episode, `check()` reads the MuJoCo state: is each named object inside the bin volume, how far apart are the two objects for next_to, which half of the table is the object on. Anything still held in a hand fails, and so does any other object pushed more than 3 cm. The model's own `done` summary is ignored. This matters because language models will happily report success; in this project the base model ended 4 episodes by claiming success on a task it had not done.

Feasible by construction, from a measured reach map

If some tasks are impossible, a planner's score mixes its own errors with the benchmark's. Rather than guess the robot's workspace, a script (`sweep_reach.py`) tried real grasps on a 9 by 21 grid of table positions, for each arm and each object type. The map was surprising: the fixed-base G1 can grasp a block only in a narrow band (x 0.28 to 0.32 m, at least 6 cm off the midline), while both arms can reach a cup placed at the middle of the table. So cups can be handed across and blocks cannot. The sampler uses this map and only ever asks for cross-table moves of cups.

The privileged expert

The **expert** is a short hand-written function that reads the true state and returns the next tool call, with a one-line reason. It knows which arm reaches what, when to hand over, and for next_to it places on a free spot computed when the task was sampled. It is "privileged" because it uses information the planner does not get as cleanly (exact state, a precomputed spot). It solves **94 of 100** eval tasks. The other 6 are physics failures, not decision errors: three cups that miss the bin, one cup that ends on the wrong side after a handover, two neighbours nudged during a place. That 94 is the ceiling for anything that learns by imitating this expert.

> **Data splits**
>
> Evaluation uses task seeds 0 to 99 and nothing trains on them. SFT data uses seeds 10000 to 10599, DAgger 30000 to 30249, GRPO 20000 and up. Seeds index the same random task distribution, so this measures how well the planner learned this family of tasks, not transfer to new objects.

---

3. Core ideas you need

Constrained decoding

A language model produces text one token at a time. **Constrained decoding** masks, at every step, the tokens that would break a given grammar, here a JSON schema of the tool call (the tool must be one of the listed names, `arm` one of left, right or both, `target` a string or three numbers). The output is then always parseable. Two points to remember: it guarantees *syntax*, never *sense* (the base model still places with an empty hand in perfectly valid JSON), and the schema itself can fix failure modes. In this project, capping the `thought` string at 200 characters stopped a looping model from running out of tokens before the JSON closed.

QLoRA: fine-tuning a 3B model in 6 GB

Full fine-tuning of 3.8 billion parameters needs tens of gigabytes for weights, gradients and optimiser state. **LoRA** freezes the original weights and learns a small low-rank correction for selected matrices:

instead of updating a matrix W, it learns two thin matrices A and B so the layer computes W x + B A x. With rank 16 on the attention and MLP projections of the language model, that is 29.9 million trainable parameters, 0.79% of the model. **QLoRA** also stores the frozen base in 4-bit NF4, a number format designed for normally distributed weights, which brings the model itself to roughly 2.5 GB. Gradients flow through the 4-bit weights into the adapters only. The vision tower stays frozen: the camera frames are not the problem, the decisions are.

Supervised fine-tuning with the loss on the answer only

Each training row is a full prompt (system text, image, state message) followed by the expert's JSON answer. The model is trained with ordinary next-token cross-entropy, but only on the answer tokens. The prompt is about 1,300 tokens and the answer about 50; without masking, most of the gradient would go into predicting the system prompt the model is given anyway. A related memory trick: the trainer asks the model for logits only at the answer positions. Full-vocabulary logits (151,000 entries) over the whole prompt cost 0.9 GB on their own and did not fit.

Compounding errors and DAgger

Supervised fine-tuning on expert demonstrations is **behaviour cloning**. Its weakness is well known: the training data only contains states on the expert's own path. When the learned planner makes one small mistake (a handover cup set down 1 cm off, a place that fails in physics), it lands in a state it has never seen, makes a worse decision, and drifts further. Errors compound over the episode.

DAgger (Dataset Aggregation, Ross et al. 2011) fixes the data rather than the model. Let the learned planner drive. At every state it reaches, ask the expert what it would do there, and record that as the training target, while the planner's own action is the one actually executed. Retrain on old plus new data. The new data covers exactly the states the planner gets itself into. This needs an expert that can be queried in any state, which a state-reading function can and a human demonstrator usually cannot, and that is why DAgger is cheap here.

Reinforcement learning on the planner, and GRPO

Imitation copies the expert; reinforcement learning optimises the outcome directly. The planner is a policy over tokens. A **policy gradient** raises the log-probability of sampled outputs in proportion to how much better than expected they turned out (the **advantage**). The classic way to estimate "expected" is a learned value network (PPO). **GRPO** (Group Relative Policy Optimisation, from DeepSeekMath, used in DeepSeek-R1) drops the value network: sample a *group* of G answers to the same prompt, score each, and use each score minus the group mean, divided by the group standard deviation, as its advantage. Better than its siblings means pushed up, worse means pushed down.

Here a "sample" is a whole episode in the real agent loop, so the reward can come from physics. Two consequences follow directly from the formula. If all G samples score the same, every advantage is zero and the group teaches nothing; strong policies produce many such groups. And with one update

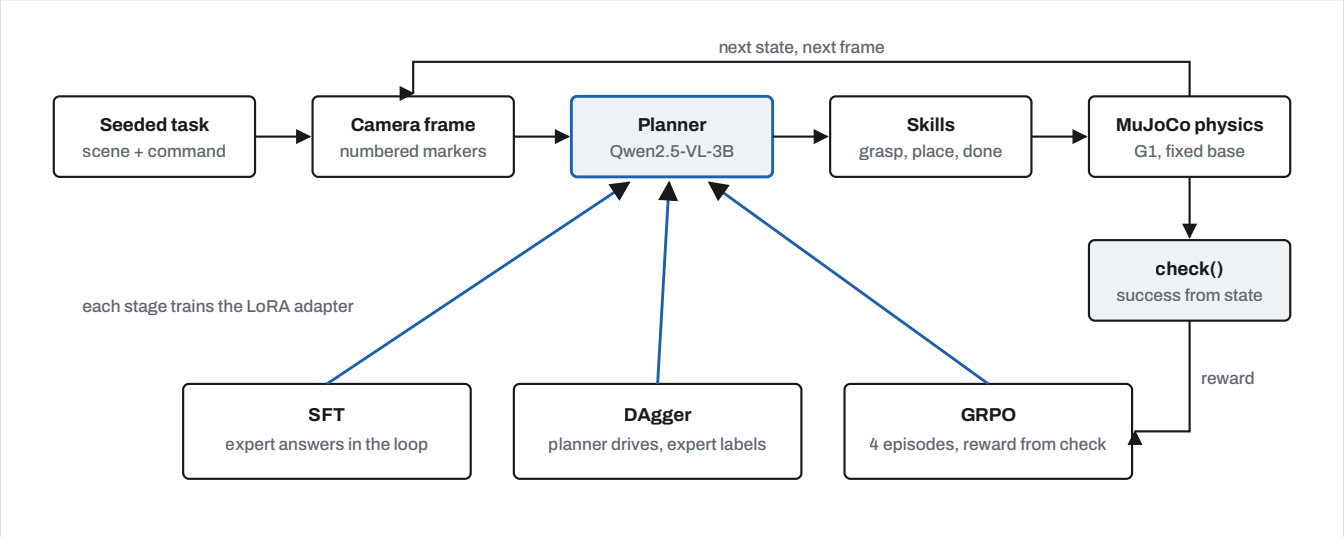
per fresh batch (fully on-policy), the PPO probability ratio is exactly 1 and the GRPO loss reduces to minus the advantage times the mean token log-probability.

Reward design

The episode reward is `success − 0.1 × failed_calls − 0.02 × calls`. Success is the physics check. The small costs separate two successful episodes when one wasted calls, which gives a group variance even when every sample succeeds, and they never outweigh success. The reward is computed from the simulator, so it cannot be gamed by the model's text.

---

4. The pipeline, stage by stage



One agent loop is used for everything: collecting SFT data, collecting DAgger data, GRPO rollouts and evaluation. Only who answers and what is recorded changes, so training prompts are byte-for-byte what the planner sees at test time.

Serving every model the same way

`serve_vlm.py` loads Qwen2.5-VL-3B-Instruct in 4-bit, optionally with a LoRA adapter, behind the same small Ollama-style `/api/chat` interface the agent always used. JSON-schema decoding is done by lm-format-enforcer, and `--greedy` makes evaluation deterministic. Base, SFT, DAgger and GRPO are all scored through this one server, so they differ only in their weights.

Stage 1: SFT data straight from the agent loop

`collect_sft.py` runs the real agent loop, but a stand-in object answers instead of the VLM: it calls the expert on the current true state and records the exact messages the planner would have received. Episodes are kept only if they succeed in physics. 575 of 600 training episodes passed, giving 2,432 steps. Using the real loop rather than writing prompts by hand removes any mismatch between

training and test formats, including the marker list, action history and the wording of error messages.

Stage 2: SFT training

setting	value	why
base	Qwen2.5-VL-3B-Instruct, 4-bit NF4, double quantisation, bf16 compute	fits 6 GB with room for activations
adapter	LoRA rank 16, alpha 32, dropout 0.05, q k v o gate up down of the language model	vision tower frozen; 0.79% of weights train
batch	1 row, gradient accumulation 8	multimodal rows differ in length
schedule	lr 2e-4, cosine, 5% warm-up, 2 epochs (550 steps)	about 4 hours at 25 s per step
validation	10% of episodes held out by seed, best checkpoint kept	steps of one episode are near duplicates; splitting by step would leak
context	system prompt + newest state message only	the state message already holds the history; 1.3k instead of 4.5k tokens

Validation loss went from 0.027 after 100 steps to 0.015 at the best checkpoint (step 500).

Stage 3: DAgger

`collect_dagger.py` serves the SFT adapter at temperature 0.2, lets it drive 250 training episodes, and asks the expert for a label at every state. Steps where the expert would call done although the task is not solved are dropped. The policy solved 219 of 250 episodes and disagreed with the expert on 198 steps; 1,097 labelled steps were added. Training continued from the SFT adapter on all 3,529 rows for one epoch at lr 1e-4 (about 2.8 hours).

Stage 4: GRPO

`grpo_train.py` loads the DAgger adapter as trainable and runs the simulator in the same process. For each training task it plays 4 episodes at temperature 1.0 under the JSON schema, scores them with the reward from section 3, skips the group if all rewards are equal, and otherwise backpropagates minus advantage times mean token log-probability over every planner call of the episode, then takes one AdamW step (lr 2e-5, gradient clipped at 1.0). Only bin, two_bin and next_to tasks were used, the families that still had failures. 147 tasks were played, 50 produced an update, 4.9 hours.

5. Evaluating fairly

1. One harness, one set of settings

Every row of the results table was produced by the same command on the same 100 seeds, through the same server, with greedy decoding, the same schema (including the thought cap) and the same "last" context. When a setting changed during the project, the earlier models were evaluated again. That is why the base model's score in the final table is 23/100, not the 27/96 measured earlier with longer prompts: numbers from different settings are not compared.

2. Physics decides, the model does not

The verdict is `check()` on the final state, never the model's summary.

3. Compare on the same tasks, with a paired test

Two planners scored on the same 100 tasks should be compared task by task. The **exact McNemar test** looks only at discordant tasks, the ones exactly one planner solved, and asks whether the split is more lopsided than a fair coin would give. SFT versus base: 61 tasks only SFT solved, 1 only base solved, p = 3e-17. DAgger versus SFT: 11 versus 2, p = 0.022. GRPO versus DAgger: 0 versus 0, no difference at all. An unpaired comparison of 92% against 83% on 100 tasks each would look far less certain than it is.

4. Report the ceiling

The expert's 94 tells the reader how much room there was. 92 means little without it; next to 94 it says the learned planner is within two tasks of its teacher.

5. Check that the measurement itself is sound

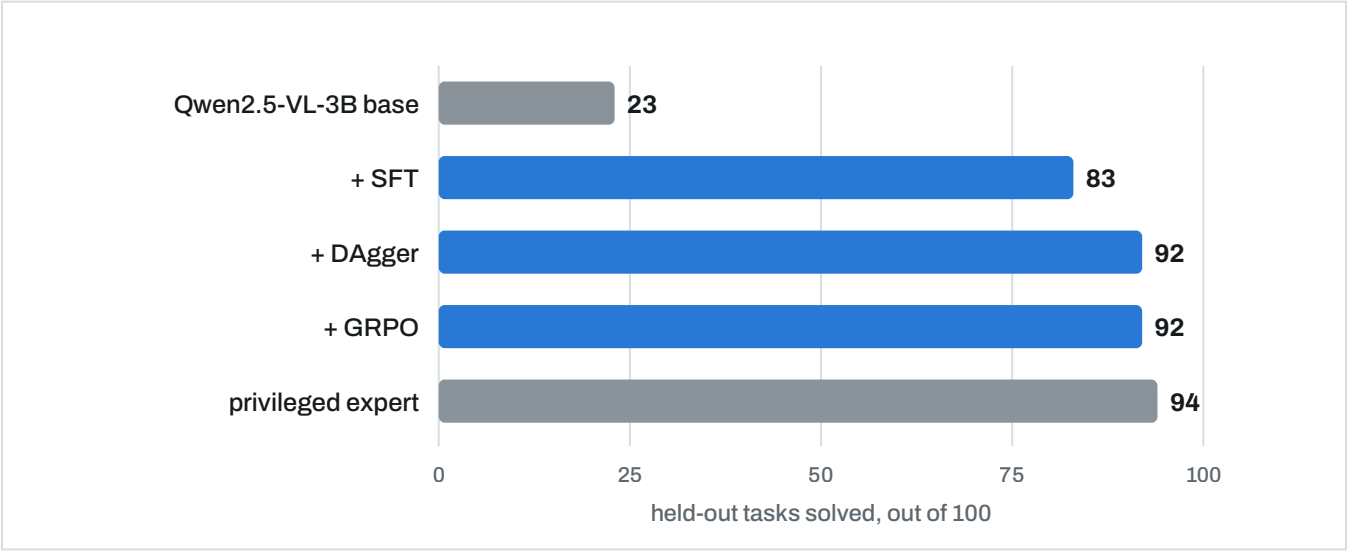
Know how exactly a run repeats. Greedy decoding on a GPU is not bit-exact across runs: for the base model, a near-tied token can flip (task 3 took a different second call in two runs and failed both times). The fine-tuned planners are confident enough that this did not show: the GRPO model reproduced the DAgger model's failure list exactly. Expect a task or two of spread for the base model.

Before trusting a surprising number, reproduce it. Two bugs in this project would have produced plausible but wrong numbers (section 6): a memory leak that killed an evaluation part way, and a renderer bug that made the robot blind. Both were caught because a number disagreed with another measurement of the same thing.

6. What happened, step by step

step	what was done	what was learned
1	Base model through Ollama on 20 tasks	6/20. Typical errors: wrong arm, place before grasp, repeating a failed call, fixing solved tasks.
2	Reach map, feasible sampler, expert	Expert 94/100. Biasing the handover point toward the receiving arm made it worse (87), so it was reverted.
3	Own HF server, base on 100 tasks	Stopped at 96 by the operating system: the server had grown to 17 GB of RAM. The JSON enforcer cached every prefix; rebuilding it per request kept the server at 2.2 GB.
4	First SFT runs	Out of memory twice: full-vocabulary logits, then 4.5k-token prompts. Fixed with answer-only logits and the "last" context.
5	SFT r1 evaluated	79/100. Seven failures were invalid JSON: the model looped one sentence in "thought" until the token limit.
6	Thought capped at 200 characters in the schema	SFT 83/100 under the cap; applied to every model.
7	First DAgger collection	Policy "success" fell to 29%. 955 of 960 states had an empty marker list: an unclosed renderer from the previous episode broke segmentation in the next. Data discarded, renderers closed every episode, agent now refuses an empty first frame.
8	DAgger collected again and trained	92/100.
9	GRPO, 50 updates	92/100, the same tasks as DAgger.

7. Final results



Each stage builds on the one above it. Grey bars are the references (untrained model, expert); blue bars are the trained planners.

planner	success	bin	two_bin	next_to	side	absent	already	failed calls	calls per solve
Qwen2.5-VL-3B base	23/100	13/28	0/16	1/25	0/16	1/4	8/11	418	5.09
SFT	83/100	22/28	11/16	20/25	15/16	4/4	11/11	46	4.07
SFT + DAgger	**92/100**	**26/28**	**14/16**	**22/25**	**15/16**	**4/4**	**11/11**	**28**	**4.15**
SFT + DAgger + GRPO	92/100	26/28	14/16	22/25	15/16	4/4	11/11	27	4.14
privileged expert	94/100	27/28	14/16	24/25	14/16	4/4	11/11	n/a	4.29

How to read this result

- **SFT does most of the work.** From 23 to 83, and failed calls from 418 to 46. The families that need reasoning about arms (two_bin, next_to, side) go from almost zero to mostly solved.
- **DAgger closes most of the remaining gap.** Its gains are in bin and two_bin, the families with handovers, which is exactly where SFT drifted into unseen states.
- **On side tasks the planner beats its teacher** (15 against 14): one of the expert's physics failures happens not to occur on the planner's slightly different trajectory. With one task in it, this is noise, not skill.
- **Latency** is 4 to 5 s per call for every model on the 6 GB GPU (HF transformers with constrained decoding). Fine-tuning did not slow the planner; it did reduce the number of calls per solved task from 5.1 to 4.1.

8. How the planners fail

The base model

The most common failed calls in its 100 episodes:

error returned by the skill	count	meaning
not holding anything	89	place with an empty hand, usually the other arm
refused: identical action already failed	67	the loop breaker stopping a repeat of a failing call
aborted: repeated the same action	56	the third identical call ends the episode
bad arguments (missing target)	44	valid JSON that omits a required argument
outside the arm's workspace, could not reach, hand not close enough	70+	grasping with the arm on the wrong side

50 of its 77 failed episodes end with an object still in a hand. It never uses the handover, so two_bin and side are 0 out of 32. And it does not stop: on all 11 "already" tasks and all 4 "absent" tasks its first call is a grasp.

After SFT

The first SFT model (before the thought cap) failed 21 tasks, 4 of which the expert also fails. Its 17 own failures fall into three groups: runaway thoughts that never close the JSON (fixed by the cap), confusion after a handover (placing at "left" instead of the bin, sending the cup back across), and next_to spots chosen next to a third object, which the place skill then shifts 17 to 31 cm away to avoid dropping onto it.

After DAgger

Eight failures, four of which the expert shares. The four that are the planner's own: two next_to placements, where it must infer a free spot from marker coordinates while the expert reads a precomputed one, and two recoveries after a failed physical place, where it called done or retried the same grasp. These are the hardest decisions in the suite and they are rare.

9. Why GRPO changed nothing

The GRPO adapter differs from the DAgger adapter by 2.5% in weight norm, and its greedy decision is identical on all 100 eval tasks. It is tempting to call that a bug. The training log says otherwise.

1. **There was almost nothing left to win.** After DAgger only 4 failures were the planner's own. Even a perfect fix moves 92 to 96, which a 100-task evaluation can barely distinguish.
2. **Most groups carried no signal.** 97 of 147 groups (66%) had four identical rewards: the planner was confident and consistent, so the group-normalised advantage was zero. GRPO learns only from disagreement among samples.
3. **The remaining errors live in rare states.** Choosing a free next_to spot near a crowded corner, or recovering after a failed place, comes up in a few percent of random training tasks. Fifty updates spread over random tasks barely visit them.

What would test GRPO properly here: train only on next_to with more samples per task (8 or 16) so the coordinates vary, or start from a weaker model (the SFT adapter, at 83) where groups disagree more often. In this project, at this budget, the on-policy correction that worked was DAgger, because the expert could be asked directly about the states that mattered. An RL signal that has to rediscover the same answer by trial is weaker than a teacher who can simply say it.

> The general point
>
> When a good expert can be queried in any state, DAgger usually beats RL fine-tuning for imitation-sized gaps. RL earns its cost when there is no expert, or when the goal is to go beyond the expert, such as the next_to spots, where the expert's choice is not available to the planner's inputs.

10. Lessons that transfer

1. **Build the evaluation before the model.** A feasible task suite with a physics verdict and an expert ceiling made every later number interpretable.
2. **Generate training data with the deployment loop.** Collecting SFT rows from the real agent loop guaranteed the prompts matched test time exactly.
3. **Read the failures, not just the score.** Grouping the 21 SFT failures by cause pointed to two fixes (the schema cap and DAgger) that together added 13 tasks.
4. **Decoding constraints are part of the model.** A one-line schema change fixed a failure that more training would have struggled with.
5. **Distrust a number that disagrees with another measurement.** The broken DAgger run was found because 29% did not match 79%; the cause was a camera bug, not the model.

6. **Fit memory by shrinking what you compute, not just the batch.** Answer-only logits and a shorter context made QLoRA on a 3B vision-language model fit in 6 GB.
7. **Re-run baselines when settings change.** The base model's 27/96 and 23/100 are both right, under different settings; only one belongs in the table.
8. **Report a null result with its cause.** "GRPO did nothing, because 66% of groups had zero variance and only 4 fixable failures remained" is more useful than a tuned-until-it-wins number.

---

11. Interview questions

Why fine-tune a 3B model instead of prompting a larger one?

The planner runs locally on the robot's computer with no cloud dependency, at about 5 s per decision on a 6 GB GPU. A larger model would not fit. Fine-tuning took the small model from 23% to 92%, within two tasks of the expert, so the size was not the limit; the missing knowledge about this robot's arms and reach was.

What does QLoRA actually train, and how much memory does it save?

It freezes the base model in 4-bit NF4 and trains low-rank adapters (rank 16) on the language model's projection matrices: 29.9M parameters, 0.79% of the model. Gradients and optimiser state exist only for those, so a 3.8B model trains in 6 GB with gradient checkpointing.

Why put the loss on the answer tokens only?

The prompt is about 1,300 tokens and the answer about 50. Training on the prompt teaches the model to predict text it is always given and dilutes the signal from the decision. It also let the trainer compute logits only at the answer positions, which was needed to fit memory.

What problem does DAgger solve, and why did it work here?

Behaviour cloning only sees the expert's states, so after a small slip the planner is in unfamiliar states and errors compound. DAgger lets the learner drive and has the expert label the states it actually visits. It worked here because the expert is a function of the state, so labelling any state is free. It added 11 tasks and lost 2 ($p = 0.022$).

Explain GRPO in two sentences.

Sample a group of answers to the same prompt, score each, and use each score's distance from the group mean (divided by the group's standard deviation) as its advantage. Then push up the log-probability of above-average answers and push down the rest, with no value network.

Why did GRPO not improve the result?

Two thirds of groups had identical rewards and so zero advantage, and after DAgger only 4 failures were the planner's own. The update moved the adapter by 2.5% without changing any greedy decision. A targeted run (next_to only, larger groups) or a weaker starting policy would be the proper test.

How do you know 92 versus 83 is real and not noise?

Both were evaluated on the same 100 tasks, so the comparison is paired. 11 tasks are solved only by DAgger and 2 only by SFT; the exact McNemar test gives $p = 0.022$. The SFT versus base comparison is 61 to 1.

What does constrained decoding guarantee?

That the output parses against the schema, every time. It does not make the call sensible: the base model produced 418 failed calls in valid JSON. It can still fix real failures, such as capping a field's length to stop a looping model from never closing the object.

How did you make sure the benchmark was fair to the model?

Every task is feasible by construction from a measured reach map, success is read from physics, eval seeds never appear in training, the expert gives a ceiling, and every model was scored through the same server and settings.

Tell me about a bug that could have invalidated the results.

The first DAgger collection showed the policy failing 71% of tasks while evaluation said 79% success. The prompts had empty marker lists: a camera renderer left alive from the previous episode broke segmentation in the next. The data was discarded, renderers are now closed every episode, and the agent raises an error if the first frame has no markers.

What would you do next?

A targeted GRPO run on next_to with 8 to 16 samples per task; give the planner a free-space hint so it is not inferring spots from coordinates alone; test transfer to new objects and command phrasings, which this benchmark does not measure; and speed up serving (vLLM or a fused 4-bit kernel) to cut the 4 to 5 s latency.

12. Glossary

Planner

	The model at the top of the hierarchy that chooses the next tool call; here Qwen2.5-VL-3B.
Tool call	A structured action such as `grasp(object, arm)` that deterministic code executes.
Set-of-Mark prompting	Drawing numbered markers on objects in the image so the model can refer to them by number or name.
Constrained decoding	Masking tokens during generation so the output always matches a grammar or JSON schema.
LoRA	Low-Rank Adaptation: training small low-rank matrices added to frozen weights.
QLoRA	LoRA on top of a base model stored in 4-bit NF4.
NF4	A 4-bit number format whose levels are spaced for normally distributed weights.
SFT	Supervised fine-tuning: next-token training on example answers.
Behaviour cloning	Imitation learning by supervised learning from state to expert action.
Compounding error	Small mistakes leading to unfamiliar states and larger mistakes over an episode.
DAgger	Dataset Aggregation: the learner drives, the expert labels the visited states, retrain on all data.
Privileged expert	A teacher with access to information the learner does not get, such as the true state.
Policy gradient	Changing a policy by raising the probability of actions in proportion to their advantage.
Advantage	How much better an outcome was than expected.
GRPO	Group Relative Policy Optimisation: advantage from the scores of a group of samples, no value network.
Zero-variance group	A GRPO group where all samples score the same; it gives no gradient.
KL penalty	A term that keeps the fine-tuned policy close to a reference model; omitted here.
Greedy decoding	Always taking the most likely token; deterministic, used for evaluation.
McNemar test	A paired test on two classifiers' discordant cases.
Handover	One arm sets the cup down at the middle of the table and the other picks it up.

13. Further reading

- Ross, Gordon, Bagnell (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. The DAgger paper.
- Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models.
- Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs.
- Shao et al. (2024). DeepSeekMath, which introduced GRPO. DeepSeek-AI (2025). DeepSeek-R1.
- Yu et al. (2025). DAPO, on dynamic sampling to drop zero-variance groups.
- Bai et al. (2025). Qwen2.5-VL Technical Report.
- Yang et al. (2023). Set-of-Mark Prompting.
- Ahn et al. (2022). Do As I Can, Not As I Say (SayCan). Liang et al. (2022). Code as Policies.
- lm-format-enforcer and Hugging Face PEFT, the libraries used for decoding and adapters.

All numbers come from the evaluation files in the repository (`posttrain/results/`, regenerated by `posttrain/summarize.py`) and the GRPO training log. Simulation only: Unitree G1 in MuJoCo 3.12, Qwen2.5-VL-3B-Instruct, transformers 4.57, PEFT 0.21, one RTX 3060 laptop GPU (6 GB).